Math	Python

$LR = \prod_i \frac{P(x_i)}{Q(x_i)}$	`p = np.array([0.1,0.1])`
	`q = np.array([0.6,0.5])`
	`def lr(p,q):`
	`  return np.prod(p/q)`

One problem with this objective function is that computers have a hard time multiplying a bunch of probabilities, since they are tiny floating-point numbers that when multiplied together create even smaller floating-point numbers. This results in numerical inaccuracies and ultimately numerical underflow, since computers have a finite range of numbers they can represent. To improve this situation, we generally use log probabilities (equivalently, log likelihoods) because the logarithm function turns tiny probabilities into large numbers ranging from negative infinity (when the probability approaches 0) up to a maximum of 0 (when the probability is 1).

Logarithms also have the nice property that $\log(a \times b) = \log(a) + \log(b)$, so we can turn multiplication into addition, and computers can handle that a lot better without risking numerical instability or overflows. We can transform the previous product log-likelihood ratio equation into this:

Table 7.5. The log-likelihood ratio in math and Python

Math	Python

$LR = \sum_i \log\left(\frac{P(x_i)}{Q(x_i)}\right)$	`p = np.array([0.1,0.1])`
	`q = np.array([0.6,0.5])`
	`def lr(p,q):`
	`  return np.sum(np.log(p/q))`

This log-probability version of the equation is simpler and better for computation, but another problem is that we want to weight individual samples differently. For example, if we sample an image of Will Smith from the data set, it should have a higher probability than an image of some less famous celebrity, since the less famous celebrity probably has fewer photos taken of them. We want our model to put more weight on learning images that are more probable out in the real world or, in other words, with respect